Predictive Analysis PS 8

Logistic Regression

ATRIJO ROY

2026-03-27

1. Problem to demonstrate the fitting of a binary response variable using logistic regression

a. Consider the Weekly data available in the ISLR package of R. Use the data from 1990 to 2008 as train data and the data on 2009-2010 as test data.

```
library(ISLR)
df = Weekly

#Converting the Direction column to a binary column
df$Direction=ifelse(df$Direction=="Up",1,0)
head(df)

##   Year   Lag1   Lag2   Lag3   Lag4   Lag5    Volume  Today Direction
## 1 1990  0.816  1.572 -3.936 -0.229 -3.484 0.1549760 -0.270         0
## 2 1990 -0.270  0.816  1.572 -3.936 -0.229 0.1485740 -2.576         0
## 3 1990 -2.576 -0.270  0.816  1.572 -3.936 0.1598375  3.514         1
## 4 1990  3.514 -2.576 -0.270  0.816  1.572 0.1616300  0.712         1
## 5 1990  0.712  3.514 -2.576 -0.270  0.816 0.1537280  1.178         1
## 6 1990  1.178  0.712  3.514 -2.576 -0.270 0.1544440 -1.372         0

#Train-Test Split
train.data=df[df$Year<=2008,-c(1,8)] #Excluding columns year and today
test.data=df[df$Year>=2009,-c(1,8)]  #Excluding columns year and today

list(dim_train=dim(train.data),dim_test=dim(test.data))

## $dim_train
## [1] 985   7
##
## $dim_test
## [1] 104   7
```

---

b. Using the training data, fit a logistic regression with Direction as the response and the five lag variables plus Volume as predictors. Are there any significant predictors? If not, which of the tests is reporting the least p value. Interpret the coefficient corresponding to that test.

```
model = glm(as.factor(Direction)~.,data=train.data,family = "binomial")
summary(model)

##
## Call:
## glm(formula = as.factor(Direction) ~ ., family = "binomial",
##     data = train.data)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.33258    0.09421   3.530 0.000415 ***
## Lag1        -0.06231    0.02935  -2.123 0.033762 *
## Lag2         0.04468    0.02982   1.499 0.134002
## Lag3        -0.01546    0.02948  -0.524 0.599933
## Lag4        -0.03111    0.02924  -1.064 0.287241
## Lag5        -0.03775    0.02924  -1.291 0.196774
## Volume      -0.08972    0.05410  -1.658 0.097240 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1354.7  on 984  degrees of freedom
## Residual deviance: 1342.3  on 978  degrees of freedom
## AIC: 1356.3
##
## Number of Fisher Scoring iterations: 4
```

The fitted model is given by,

$$\hat{E}(Y=1|x) = \frac{\exp(0.33258 - 0.06231Lag1 + 0.04468Lag2 - 0.01546Lag3 - 0.03111Lag4 - 0.03775Lag5 - 0.08972Volume)}{1 + \exp(0.33258 - 0.06231Lag1 + 0.04468Lag2 - 0.01546Lag3 - 0.03111Lag4 - 0.03775Lag5 - 0.08972Volume)}$$

where $Y = 1$, if Direction = Up

and $Y = 0$, if Direction = Down

Interpretations:

- Lag1 is the only predictor that is statistically significant at the 5% level (p-value = 0.033762). All other predictors, Lag2, Lag3, Lag4, Lag5, and Volume, have p-values greater than 0.05 and are therefore not significant at this level.
- Among the predictors, Lag1 has the smallest p-value.

- The coefficient of Lag1 is -0.06231, which means that for a one-unit increase in Lag1, the log-odds of the market going "Up" decreases by 0.06231, holding all other predictors constant.

---

c. Estimate the odds of market going "up" at median values of the predictors.

```
med=apply(train.data[,-7],2,median)
med

##     Lag1     Lag2     Lag3     Lag4     Lag5   Volume
## 0.231000 0.234000 0.231000 0.230000 0.230000 0.804848

newdata=as.data.frame(t(med))
log.odds=predict(model, newdata, type = "link")
odds=exp(log.odds)
odds

##        1
## 1.267475
```

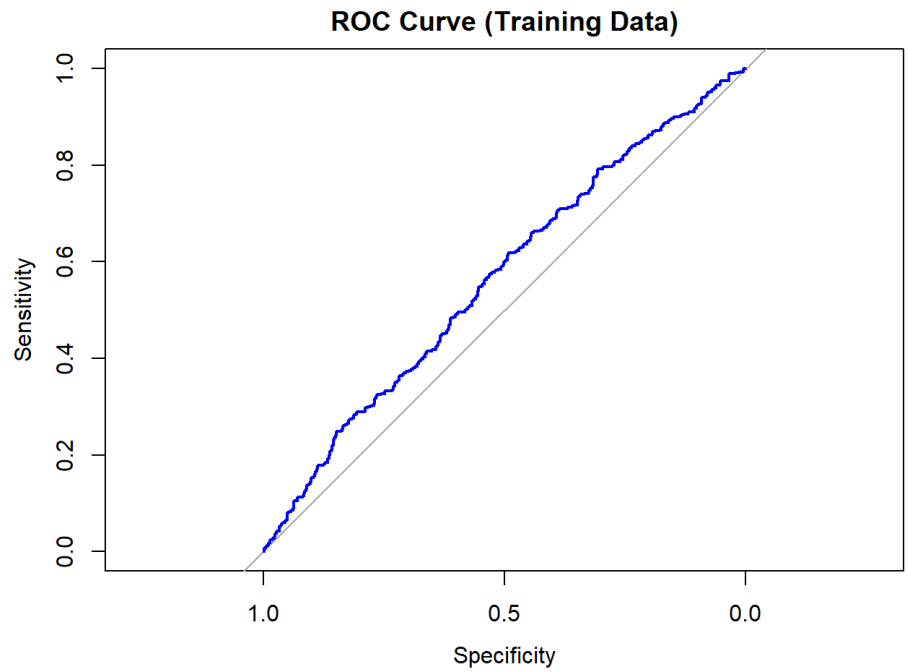
Interpretation:

The estimated odds of the market going Up (relative to Down) at the median values of the predictors is 1.267 on the average.

---

d. Based on the training sample draw the ROC curve and comment.

```
library(pROC)

## Type 'citation("pROC")' for a citation.

##
## Attaching package: 'pROC'

## The following objects are masked from 'package:stats':
##
##     cov, smooth, var

y.hat.train=predict(model,type="response")
roc.train=roc(train.data$Direction,y.hat.train)

## Setting levels: control = 0, case = 1

## Setting direction: controls < cases

plot(roc.train, col="blue", main="ROC Curve (Training Data)")
```



Interpretation:

The ROC curve lies close to the diagonal reference line, indicating that the model has poor discriminative ability. The model shows weak predictive performance on the training data.

---

e. Using the fitted model and the predictor values from the test data, find an optimal cut point that minimizes TPR(1 – FPR). Report the confusion matrix corresponding to the optimal cut-point value. From the confusion matrix compute the test error rate.

```
y.hat.test = predict(model, newdata = test.data, type = "response")
roc.test = roc(test.data$Direction, y.hat.test)

## Setting levels: control = 0, case = 1

## Setting direction: controls < cases

tpr = roc.test$sensitivities
fpr = 1 - roc.test$specificities

crit = tpr*(1-fpr)

opt.index = which.max(crit)
opt.cut = roc.test$thresholds[opt.index]
opt.cut

## [1] 0.4568085
```

The optimal cut-point is 0.4568, which means that observations with predicted probability greater than 0.4568 are classified as “Up”(=1), while those with probability less than or equal to 0.4568 are classified as “Down”(=0). This threshold maximizes the criterion TPR(1 – FPR).

```
pred.class = ifelse(y.hat.test > opt.cut, 1, 0)

confusion.mat = table(Y.hat = pred.class, Y = test.data$Direction)
confusion.mat

##      Y
## Y.hat  0  1
##     0 22 27
##     1 21 34
```

Interpretation:

- The confusion matrix shows that 22 observations were correctly classified as Down (true negatives) and 34 observations were correctly classified as Up (true positives). However, 27 observations that were actually Up were incorrectly classified as Down (false negatives), and 21 observations that were actually Down were incorrectly classified as Up (false positives).
- This indicates that the model makes a considerable number of misclassifications, particularly in identifying upward movements. While it correctly predicts some cases, its overall classification performance on the test data is moderate rather than strong.

```
test.error = mean(pred.class!=test.data$Direction)
test.error

## [1] 0.4615385
```

Interpretation:

The test error rate is 0.4615, which means that approximately 46.15% of the observations in the test dataset are incorrectly classified by the model. This indicates that the model has relatively poor predictive performance on unseen data, as nearly half of the predictions are wrong.

---

f. Is there any change in the test error rate if we have worked with only the two predictors lag 1 and lag 2?

```
m2 = glm(as.factor(Direction) ~ Lag1 + Lag2,
         data = train.data, family = "binomial")
y.hat.test.2 = predict(m2, newdata = test.data, type = "response")

roc.test.2 = roc(test.data$Direction, y.hat.test.2)

## Setting levels: control = 0, case = 1

## Setting direction: controls < cases

tpr2 = roc.test.2$sensitivities
fpr2 = 1 - roc.test.2$specificities

crit2= tpr2*(1-fpr2)

opt.cut2 = roc.test.2$thresholds[which.max(crit2)]

pred.class2 = ifelse(y.hat.test.2 > opt.cut2, 1, 0)

# Test error rate
test.error2 = mean(pred.class2 != test.data$Direction)
test.error2

## [1] 0.3557692
```

Interpretation:

- The test error rate using only Lag1 and Lag2 is 0.3558, which is lower than the previous test error rate of 0.4615. This indicates an improvement in predictive performance when using only these two predictors.
- Thus, the simpler model with Lag1 and Lag2 performs better on the test data.